

Implementations of MVCC and Garbage Collection

Muyan Zhou

May 16, 2025

1 Basic Design Ideas

- **ART:** The Adaptive Radix Tree (ART) in this project serves as a static key-value index. The key is determined by the user and maps to a corresponding row ID. The ART is constructed during the initialization phase and remains unchanged afterward.
- **Row:** The `Row` structure in `table.hpp` holds the logical contents of a row, including its ID, the parent table, a list of committed versions, and potentially an uncommitted version. Unlike Project 1, MVCC logic is embedded directly within the `Row` structure, not the ART.

At initialization, the ART is built and each `Row` is initialized. During transaction execution (queries or updates), the ART is used to locate a `Row`, and its `VersionSkipList` provides access to visible versions. When a transaction modifies a row, the new version is staged within the `Row`. This version is only made visible (inserted into the skip list) upon successful commit, maintaining consistency and isolation. If the transaction is aborted, the staged version is discarded.

2 Implementation of Each Component

2.1 SNAPSHOT Isolation MVCC

Snapshot isolation ensures that each transaction reads from a consistent snapshot of the database taken at the time the transaction starts. A transaction only sees changes committed before it began, and its own writes.

Each transaction is assigned a unique transaction ID and snapshot timestamp. Reads filter versions based on visibility—only versions committed before the transaction’s snapshot are visible. Writes are staged and only installed upon a successful commit.

At commit time, each modified row is revalidated by retrieving its latest committed version from the skip list and comparing that version’s commit timestamp against the transaction’s start timestamp. If the latest commit timestamp

exceeds the transaction's start timestamp—indicating another transaction wrote to the row after this transaction began—a write-write conflict is detected, and the transaction is aborted.

2.2 Serializable MV-OCC

Serializable isolation is achieved through Multi-Version Optimistic Concurrency Control (MV-OCC). Unlike snapshot isolation, serializable isolation also prevents anomalies such as write skew.

The implementation performs forward and backward validation at commit time:

- **Forward Check:** Ensures that data read by the transaction has not been modified by other transactions since it was read.
- **Backward Check:** Ensures that no concurrent transaction has read data that this transaction subsequently modified.

This two-phase validation guarantees serializability. If any check fails, the transaction is aborted.

2.3 Garbage Collection

MVCC systems retain multiple versions of data for isolation, but old versions must be periodically removed to reclaim memory.

Garbage collection is driven by the **TransactionManager**. It keeps track of all running transactions and identifies the oldest active transaction. Versions that are no longer visible to any running transaction—i.e., versions with commit timestamps older than the oldest read timestamp—are safe to delete.

This is implemented by:

- Tracking the minimum snapshot timestamp across all active transactions.
- Iterating over committed versions and removing those that are no longer visible to any active transaction.

Garbage collection is triggered during commits and is scoped to the rows modified by the committing transaction, ensuring it stays lightweight and incremental. To be more specific, in the **VersionSkipList** of the corresponding row, the garbage collection process involves:

- Traversing the skip list to find the version that is latest to the oldest active transaction. This part only requires a shared lock on the skip list. If there's no version that is older than the oldest active transaction, we can skip the next step.
- Removing all versions that are older than the version found in the previous step. This part requires an exclusive lock on the skip list.

As a result, the garbage collection process is relatively lightweight.